

Session 2 quiz — Intermediate Python for Finance

Ungraded in-class practice. Exam-style: code snippet + 5 true/false statements.

- **Review:** one question at a time
 - **Scope:** full DataCamp course (Ch 1–4)
 - **Plan:** 12 questions (Q11–Q12 are tricky review)
-

Question 1 (Chapter 1 — `datetime()` constructor)

Consider the following code:

```
from datetime import datetime
black_monday = datetime(1987, 10, 19)
print(black_monday)
```

- A. `print(black_monday)` displays exactly the string `'1987-10-19'`.
 - B. `datetime(1987, 19, 10)` creates the same `datetime` object as `datetime(1987, 10, 19)`.
 - C. `black_monday.month` is equal to 10.
 - D. The type of `black_monday` is `<class 'datetime.date'>`.
 - E. `black_monday.day` is equal to 19.
-

Question 2 (Chapter 1 — `strptime()` and format codes)

Consider the following code:

```
from datetime import datetime
text = "10/27/1997"
format_str = "%m/%d/%Y"
sell_date = datetime.strptime(text, format_str)
print(sell_date)
```

- A. `datetime.strptime(text, format_str)` returns a string.
 - B. `sell_date.day` is equal to 27.
 - C. Replacing `format_str` with `"%d/%m/%Y"` would produce a `datetime` with month equal to 10.
 - D. `sell_date.year` is equal to 1997.
 - E. `sell_date.month` is equal to 10.
-

Question 3 (Chapter 1 — `strftime()` and `datetime` attributes)

Consider the following code:

```
from datetime import datetime
crash = datetime(1929, 10, 29)
label = crash.strftime("%Y-%m-%d")
print(label)
```

- A. `crash.strftime("%Y-%m-%d")` returns a `datetime` object.
- B. `label` is equal to `'1929-10-29'`.

- C. The format string "%Y-%m-%d" includes the code %b for abbreviated month name.
 - D. crash.hour is equal to 12.
 - E. crash.month is equal to 10.
-

Question 4 (Chapter 1 — Comparing datetimes)

Consider the following code:

```
from datetime import datetime
asian_crisis = datetime(1997, 7, 2)
world_mini_crash = datetime(1997, 10, 27)
print(asian_crisis < world_mini_crash)
print(asian_crisis == world_mini_crash)
```

- A. Datetimes can be compared with <, >, ==, and other comparison operators.
 - B. asian_crisis == world_mini_crash evaluates to True.
 - C. datetime(1997, 7, 2) < datetime(1997, 7, 2) evaluates to True.
 - D. asian_crisis > world_mini_crash evaluates to True.
 - E. asian_crisis < world_mini_crash evaluates to True.
-

Question 5 (Chapter 2 — == vs = and cross-type equality)

Consider the following code:

```
x = 13
print(3 == 3.0)
print(3 == '3')
print(x == 13)
```

- A. The operator == assigns the value on the right to the name on the left.
 - B. print(x == 13) outputs True.
 - C. 3 == 3 evaluates to False.
 - D. print(3 == '3') outputs True.
 - E. print(3 == 3.0) outputs True.
-

Question 6 (Chapter 2 — Order operators)

Consider the following code:

```
print(3 < 4)
print('e' <= 'a')
print(4 >= 4)
```

- A. print(4 >= 4) outputs False.
- B. print('e' <= 'a') outputs True.
- C. The expression 'a' < 23 raises a TypeError.

D. `print(3 < 3)` outputs True.

E. `print(3 < 4)` outputs True.

Question 7 (Chapter 2 — Boolean operators and truthiness)

Consider the following code:

```
price = 56.88
volume = 0
print(True and False)
print(False or True)
print(not volume)
print(bool(price))
```

A. The empty string "" evaluates as True in a Boolean context.

B. `print(not volume)` outputs True.

C. `print(True and False)` outputs True.

D. `print(bool(price))` outputs False.

E. `print(False or True)` outputs True.

Question 8 (Chapter 3 — DataFrame from a dict)

Consider the following code:

```
import pandas as pd
data = {'Bank Code': ['BA', 'AAD', 'BA'],
        'Account#': ['ajfdk2', '1234nmk', 'mm3d90'],
        'Balance': [1222.00, 390789.11, 13.02]}
df = pd.DataFrame(data=data)
print(df.shape)
print(df.loc[0, 'Balance'])
```

A. df has 2 rows.

B. `df.loc[0, 'Balance']` is equal to 1222.00.

C. `df.shape` is (3, 2).

D. 'Bank Code' is a column name in df.

E. `df.shape` is (3, 3).

Question 9 (Chapter 3 — DataFrame from list of lists + columns=)

Consider the following code:

```
import pandas as pd
data = [['BA', 'ajfdk2', 1222.00],
        ['AAD', '1234nmk', 390789.11],
        ['BA', 'mm3d90', 13.02]]
columns = ['Bank Code', 'Account#', 'Balance']
df = pd.DataFrame(data=data, columns=columns)
```

```
print(df.loc[1, 'Balance'])
print(list(df.columns))
```

- A. Without the `columns=` argument, pandas would still name the columns 'Bank Code', 'Account#', and 'Balance'.
 - B. `df.loc[1, 'Balance']` is equal to 390789.11.
 - C. `df.shape` is (3, 2).
 - D. `list(df.columns)` equals ['Bank Code', 'Account#', 'Balance'].
 - E. `df.shape` is (3, 3).
-

Question 10 (Chapter 4 — `.head()`, `.tail()`, and `.describe()`)

Consider the following code:

```
import pandas as pd
prices = pd.Series([247.74, 258.44, 245.52, 246.88, 224.37])
print(len(prices.head(2)))
print(prices.tail().iloc[-1])
print(prices.describe()['count'])
```

- A. `prices.describe()['count']` is equal to 5.0.
 - B. `prices.tail().iloc[-1]` is equal to 247.74.
 - C. `len(prices.head(2))` is equal to 2.
 - D. `prices.head()` without an argument returns 5 rows by default.
 - E. `prices.describe()` only works on string columns.
-

Question 11 (Review — format-string trap: `%m` vs `%M`)

Consider the following code:

```
from datetime import datetime
text = "10/27/1997"
parsed = datetime.strptime(text, "%M/%d/%Y")
print(parsed.month)
print(parsed.minute)
```

- A. In a format string, `%M` stands for the month number (01–12).
 - B. `parsed.day` is equal to 27.
 - C. `parsed.month` is equal to 10.
 - D. Replacing `"%M"` with `"%m"` in the format string would make `parsed.month` equal to 10.
 - E. `parsed.minute` is equal to 10.
-

Question 12 (Review — DataFrame default column names)

Consider the following code:

```
import pandas as pd
data = [['BA', 'ajfdk2', 1222.00],
        ['AAD', '1234nmk', 390789.11],
        ['BA', 'mm3d90', 13.02]]
df = pd.DataFrame(data=data)
print(list(df.columns))
print(df.iloc[0, 0])
```

- A. `df.iloc[0, 0]` is equal to 'BA'.
- B. `df.shape` is (2, 3).
- C. `list(df.columns)` equals [0, 1, 2].
- D. `df.columns[0]` is equal to 'Bank Code'.
- E. Passing `columns=['Bank Code', 'Account#', 'Balance']` to `pd.DataFrame` would assign those column names.